

ASSIGNMENT # 02

PART - A

SUBMITTED BY

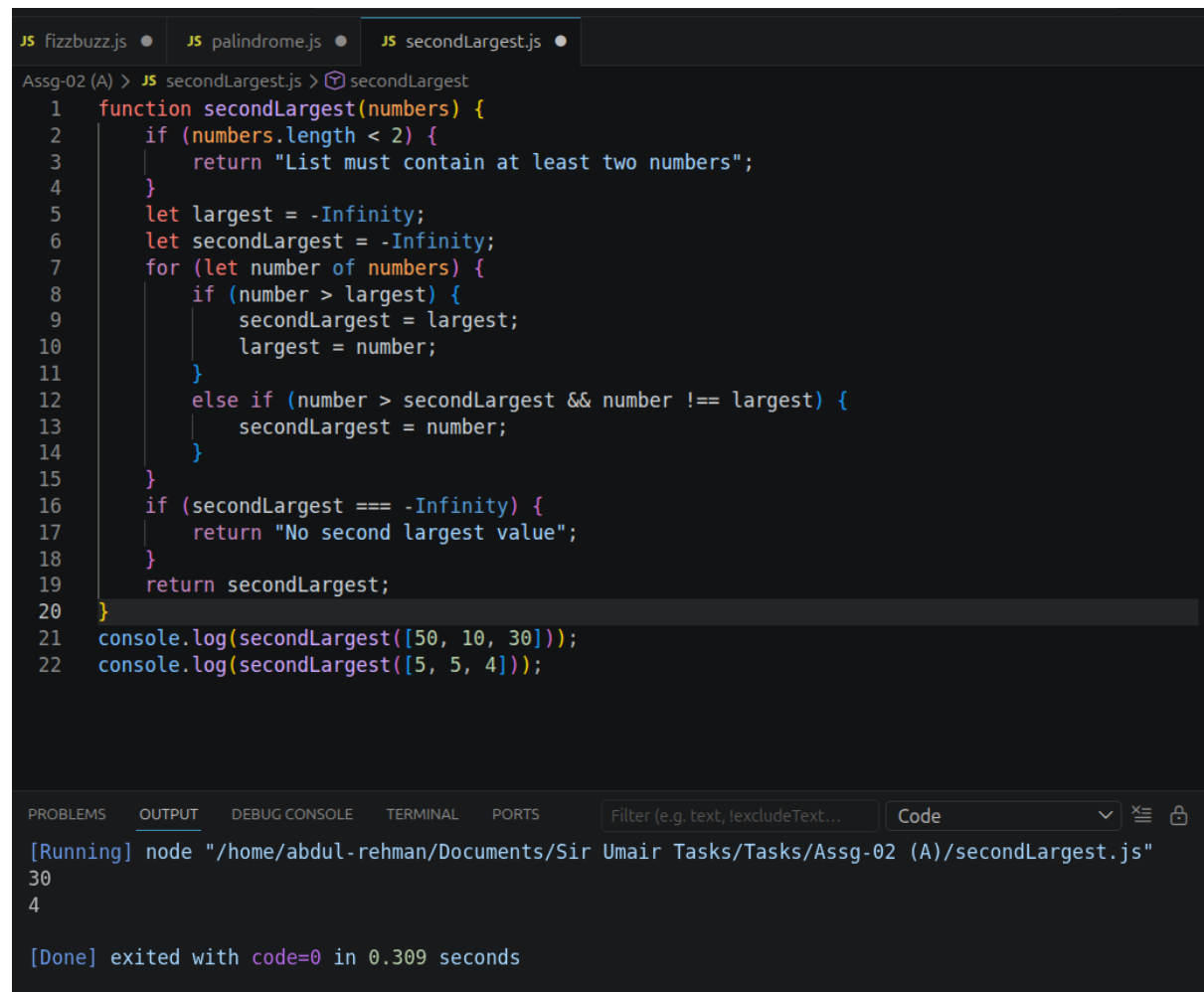
ABDUL REHMAN

SUBMITTED TO

Sir. Umair

Part A - Real code, edge cases and complexity

Second largest - fix your own bug. Write a function `secondLargest(numbers)` that returns the second largest value in a list. Test it on `[50, 10, 30]` and `[5, 5, 4]`. Note: your plain-English version in Assignment 1 had a subtle bug - it would return 50 for `[50, 10, 30]`. Work out why, and make your code return the correct answer. Then write one sentence on the time complexity (Big-O) of your solution.



```
JS fizzbuzz.js JS palindrome.js JS secondLargest.js
Assg-02 (A) > JS secondLargest.js > secondLargest
1 function secondLargest(numbers) {
2   if (numbers.length < 2) {
3     return "List must contain at least two numbers";
4   }
5   let largest = -Infinity;
6   let secondLargest = -Infinity;
7   for (let number of numbers) {
8     if (number > largest) {
9       secondLargest = largest;
10      largest = number;
11    }
12    else if (number > secondLargest && number !== largest) {
13      secondLargest = number;
14    }
15  }
16  if (secondLargest === -Infinity) {
17    return "No second largest value";
18  }
19  return secondLargest;
20 }
21 console.log(secondLargest([50, 10, 30]));
22 console.log(secondLargest([5, 5, 4]));

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, excludeText...) Code
[Running] node "/home/abdul-rehman/Documents/Sir Umair Tasks/Tasks/Assg-02 (A)/secondLargest.js"
30
4
[Done] exited with code=0 in 0.309 seconds
```

Code Explanation

Function / Section	Code Segment	Description
Function Start	function secondLargest(numbers)	Defines a function that accepts an array of numbers and returns the second largest value.
Length Check	if (numbers.length < 2)	Checks whether the array contains at least two elements. If not, finding a second largest value is not possible.
Largest Variable	let largest = -Infinity;	Stores the largest value found in the array it is initialized with Infinity so any number can replace it.

Second Largest Variable	let secondLargest = -Infinity;	Stores the second largest value found in the array it is also initialized with Infinity.
Loop	for (let number of numbers)	Iterates through each number in the array one by one.
Largest Update Condition	if (number > largest)	Checks whether the current number is greater than the current largest value.
Largest Update	secondLargest = largest; largest = number;	Moves the previous largest value to secondLargest and updates largest with the current number.
Second Largest Update Condition	else if (number > secondLargest && number !== largest)	Checks whether the current number is smaller than the largest value but greater than the current second largest value.
Second Largest Update	secondLargest = number;	Updates the second largest value with the current number.
Duplicate Handling	number !== largest	Prevents duplicate largest values from being considered as the second largest value.
Return Value	return secondLargest;	Returns the second largest number after all elements have been processed.

The previous algorithm failed because it did not correctly update the second largest value when a larger number was found later in the list in the example [50, 10, 30] the correct second largest number is 30 and the old logic could incorrectly keep 50 as the second largest value the improved solution updates both largest and second largest values properly so it always returns the correct answer and the time complexity of this solution is $O(n)$ because each element of the array is checked only one time.

Palindrome - two ways. Write isPalindrome(word) twice: (a) by reversing the string and comparing, and (b) using two pointers (one starting from each end, moving inward) - without building a reversed copy. Make both handle capital letters and spaces, so "Race car" counts as a palindrome. Then answer: which version uses less memory, and why?

```
JS fizzbuzz.js JS palindrome.js X
Assg-02 (A) > JS palindrome.js > ...
15 function isPalindromeTwoPointers(word) {
22     let right = cleanedWord.length - 1;
23
24     while (left < right) {
25
26         if (cleanedWord[left] !== cleanedWord[right]) {
27             return false;
28         }
29
30         left++;
31         right--;
32     }
33
34     return true;
35 }
36
37 console.log(isPalindromeReverse("Race car"));
38 console.log(isPalindromeTwoPointers("Race car"));
39
40 console.log(isPalindromeReverse("Hello"));
41 console.log(isPalindromeTwoPointers("Hello"));

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, excludeText...) Code
[Running] node "/home/abdul-rehman/Documents/Sir Umair Tasks/Tasks/Assg-02 (A)/palindrome.js"
true
true
false
false

[Done] exited with code=0 in 0.489 seconds
```

Code Explanation

Function / Section	Code Segment	Description
Function Start (Method 1)	function isPalindromeReverse(word)	Defines a function that checks whether a word is a palindrome by reversing the string and comparing it with the original string.
Cleaning the Input	word.toLowerCase().replaceAll(" ", "")	Converts the word to lowercase and removes all spaces to ensure a consistent comparison.
Reversing the String	cleanedWord.split("").reverse().join("")	Converts the string into an array, reverses the characters, and joins them back into a string.
Comparison	return cleanedWord === reversedWord;	Compares the cleaned string with its reversed version and returns true if both are identical.

Function Start (Method 2)	<code>function isPalindromeTwoPointers(word)</code>	Defines a function that checks whether a word is a palindrome using the two pointer technique.
Cleaning the Input	<code>word.toLowerCase().replaceAll(" ", "")</code>	Converts the word to lowercase and removes spaces before performing comparisons.
Pointer Initialization	<code>let left = 0;let right = cleanedWord.length - 1;</code>	Creates two pointers one at the beginning and one at the end of the string.
Character Comparison Loop	<code>while (left < right)</code>	Continues comparing characters until the pointers meet in the middle.
Mismatch Check	<code>if (cleanedWord[left] !== cleanedWord[right])</code>	Checks whether the characters at both pointers are different if they are different the function returns false.
Moving the Pointers	<code>left++;right--;</code>	Moves both pointers toward the centre after each successful comparison.
Return Value	<code>return true;</code>	Returns true if all character comparisons match indicating that the word is a palindrome.

The two pointer version uses less memory because it compares characters directly from both ends of the string without creating a reversed copy the reverse method creates an additional string in memory while the two pointer approach only uses a few variables therefore the two pointer solution is more memory efficient.

FizzBuzz - as real code, then made flexible. First, write FizzBuzz as actual running code (1–100). Then upgrade it: make the rules configurable - the function should take the words and their divisors as parameters, so the same function could produce "Fizz/Buzz/Bazz" for 3/5/7 without you rewriting the logic.

```
JS fizzbuzz.js •
Assg-02 (A) > JS fizzbuzz.js > FlexibleFizzBuzz
1 // Normal FizzBuzz
2 for (let i = 1; i <= 100; i++) {
3     if (i % 3 === 0 && i % 5 === 0) {
4         console.log("FizzBuzz");
5     }
6     else if (i % 3 === 0) {
7         console.log("Fizz");
8     }
9     else if (i % 5 === 0) {
10        console.log("Buzz");
11    }
12    else {
13        console.log(i);
14    }
15 }
16 // Flexible FizzBuzz
17 function flexibleFizzBuzz(limit, rules) {
18     for (let i = 1; i <= limit; i++) {
19         let output = "";
20         for (let rule of rules) {
21             if (i % rule.divisor === 0) {
22                 output += rule.word;
23             }
24         }
25         console.log(output || i);
26     }
27 }
28 console.log("Flexible FizzBuzz Output:");
29 flexibleFizzBuzz(30, [
30     { divisor: 3, word: "Fizz" },
31     { divisor: 5, word: "Buzz" },
32     { divisor: 7, word: "Bazz" }
33 ]);
```

Code Explanation

Function / Section	Code Segment	Description
Normal FizzBuzz	for (let i = 1; i <= 100; i++)	Uses a loop to iterate through numbers from 1 to 100.
Checking Divisibility by 3 and 5	if (i % 3 === 0 && i % 5 === 0)	Checks whether a number is divisible by both 3 and 5 if true "FizzBuzz" is displayed.
Checking Divisibility by 3	else if (i % 3 === 0)	Checks whether a number is divisible only by 3 if true "Fizz" is displayed.
Checking Divisibility by 5	else if (i % 5 === 0)	Checks whether a number is divisible only by 5 if true "Buzz" is displayed.
Printing the Number	else	If the number is not divisible by 3 or 5 the number itself is displayed.
Flexible FizzBuzz Function	function flexibleFizzBuzz(limit, rules)	Defines a reusable FizzBuzz function that accepts custom divisibility rules.

Loop Through Numbers	for (let i = 1; i <= limit; i++)	Iterates through all numbers from 1 up to the specified limit.
Loop Through Rules	for (let rule of rules)	Checks each rule provided in the rules array.
Divisibility Check	if (i % rule.divisor === 0)	Verifies whether the current number is divisible by the rules divisor.
Output Generation	output += rule.word;	Adds the corresponding word to the output string when a rule matches.
Display Result	`console.log(output	
Flexibility of the Solution	rules parameter	Allows new divisibility rules to be added without modifying the main program logic making the solution reusable and extensible.

Output:

Output	Output	Output	Output	Fizz Buzz
1	1	64	Buzz	Fizz
2	Fizz	Buzz	Fizz	Buzz
Fizz	34	Fizz	97	Fizz
4	Buzz	67	98	Bazz
Buzz	Fizz	68	Fizz	29
Fizz	37	Fizz	Buzz	FizzBuzz
7	38	Buzz	Flexible FizzBuzz Output:	
8	Fizz	71	1	=== Code Execution Successful ===
Fizz	Buzz	Fizz	2	
Buzz	41	73	Fizz	
11	Fizz	74	4	
Fizz	43	FizzBuzz	Buzz	
13	44	76	Fizz	
14	FizzBuzz	77	Bazz	
FizzBuzz	46	Fizz	8	
16	47	79	Fizz	
17	Fizz	Buzz	Buzz	
Fizz	49	Fizz	11	
19	Buzz	82	Fizz	
Buzz	Fizz	83	13	
52	53	Fizz	Bazz	
Fizz	56	Buzz	FizzBuzz	
22	Fizz	86	16	
23	Buzz	Fizz	17	
Fizz	58	88	Fizz	
Buzz	Fizz	89	19	
26	59	FizzBuzz	Buzz	
Fizz	61	91	FizzBazz	
28	FizzBuzz	92	22	
29	62	Fizz	23	
FizzBuzz	Fizz	94	Fizz	
31	64	Buzz	Buzz	
32				